All of the coding implementations are included in the Jupyter notebook. We set seed = 42 in the whole project.

Part 1: Mixture of Bernoulli products

(a) To describe the complete data, we introduce the following latent variable $\mathbf{z} = (z_1, z_2, ..., z_K)$, such that $\sum_{k=1}^{K} z_k = 1$ and $z_k \in \{0, 1\}$ with $\mathbb{P}(z_k = 1) = \pi_k$ for each k. And the generation procedure could be describe as one probability $f(x \mid z_k) = p(x \mid \mu_k)$. Hence, the complete data likelihood is

$$p(x, \mathbf{z}|\mu, \pi) = \prod_{k=1}^{K} \pi_k^{z_k} p(x \mid \mu_k)^{z_k}.$$

while the log-likelihood is

$$\log p(x, \mathbf{z}|\mu, \pi) = \sum_{k=1}^{K} z_k \left(\log(\pi_k) + \log p(x \mid \mu_k)\right) = \sum_{k=1}^{K} z_k \left(\log(\pi_k) + \sum_{d=1}^{D} x_d \log \mu_{k,d} + (1 - x_d) \log(1 - \mu_{k,d})\right).$$

To perform EM-algorithm, we shall study the posterior of the latent variable $\mathbf{z}$, which is

$$p(z_k = 1 \mid x_i) \propto p(x_i \mid z_k = 1) p(z_k = 1) = \pi_k p(x_i \mid \mu_k),$$

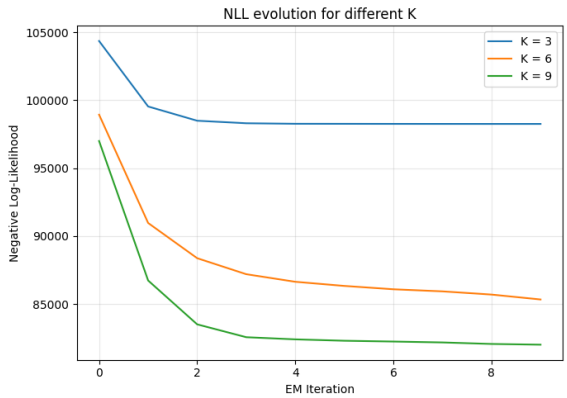
where x_i is one sample from given distribution. So we have E-Step:

$$\gamma_{ik} = p(z_k = 1 \mid x_i) = \frac{\pi_k p(x_i \mid \mu_k)}{\sum_{k=1}^{K} \pi_k p(x_i \mid \mu_k)}, \text{for any k.}$$

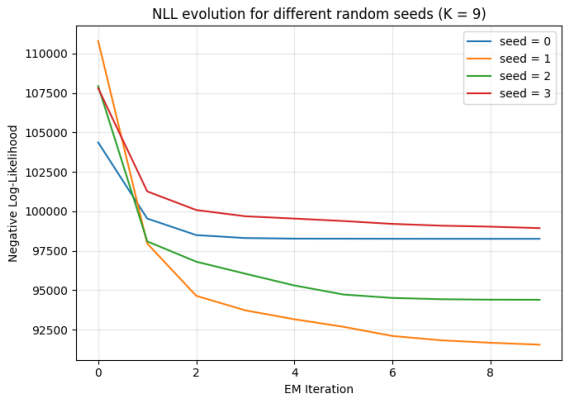
Taking the expectation w.r.t posterior, and note that $\mathbf{x} = (x_1, x_2, ...x_N)$. Suppose N samples are given, we have

$$\begin{aligned}
\mathbb{E}_{z \sim p(\mathbf{z}|\mathbf{x},\mu)}[\log p(\mathbf{x}, \mathbf{z}|\mu, \pi)] &= \sum_{n=1}^{N} \sum_{k=1}^{K} \gamma_{nk} \left(\log(\pi_k) + \log p(x \mid \mu_k)\right) \\
&= \sum_{n=1}^{N} \sum_{k=1}^{K} \gamma_{nk} \left(\log(\pi_k) + \sum_{d=1}^{D} x_{nd} \log \mu_{k,d} + (1 - x_{nd}) \log(1 - \mu_{k,d})\right).
\end{aligned}$$

We maximize (M-Step) this expectation by first using Lagrangian to update every $\pi_k = \sum_{n=1}^{N} \gamma_{nk}/N$, and taking the derivative for each $\mu_{k,d}$, we update then that $\mu_{k,d} = \sum_{n=1}^{N} \gamma_{nk} x_{nd} / \sum_{n=1}^{N} \gamma_{nk}$.
(b) Concerning the reason why we operate in the log-probability space, the computation of γ_{ik} involves a considerable number of multiplications, which can be computationally expensive. Transforming multiplications into summations simplifies the computation and improves numerical stability. In addition to this, this method may avoid issue in numerical underflow.
(c) We conduct experiments by varying two hyperparameters, namely the number of mixture components K and the initial mean μ. We observe that increasing K leads to a substantial improvement in model fit, as reflected by a lower final negative log-likelihood. However, larger values of K also result in slower convergence. On the other hand, different random initializations of μ lead to huge fluctuations in the NLL curves, while some seeds have converged in 10 steps but the others are not convergent, meaning that EM-algorithm is sensitive to initial start.



(a) NLL evolution for different values of K.



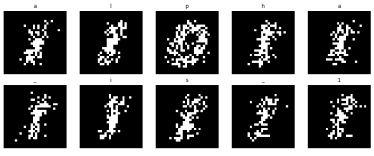
(b) NLL evolution for different random seeds.

Figure 1: Training loss evolution under different hyperparameter settings.

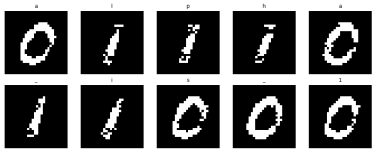
(d) The loss decreases monotonically in all experiments, as EM guarantees a non-decreasing log-likelihood at each iteration, and thus a decreasing negative log-likelihood. However, not all runs converge within the given number of iterations. For example, when seed = 1, the loss does not clearly stabilise within 10 iterations, suggesting that more iterations may be required under certain initializations. Moreover, different initial means μ lead to different convergence behaviours. This highlights the sensitivity of EM to initialization and its tendency to converge to different local optima.

(e) Due to symmetry, it suffices to discuss a generic component μ_k. In the MNIST setting, μ_k is a vector whose entries correspond to the Bernoulli parameters of each pixel. More specifically, each element of μ_k represents the probability that a given pixel is black under component k. Therefore, μ_k can be interpreted as the prototype or average image associated with a particular digit class learned by the mixture model.

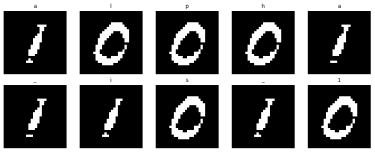
(f) The displayed images are not clearly distinguishable because the model assumes pixel-wise independence under the mean-field approximation, whereas real digits exhibit strong spatial correlations. Ignoring these dependencies leads to noisy samples and limits the model's ability to capture meaningful structures. In addition, converting grayscale MNIST images where all pixels take values (from 0 to 1) into binary (from 0 and 1) form causes significant information loss, which further degrades the visual quality.



$\alpha = 1$. Generate from Mixture Bernoulli directly



$\alpha = 10$.



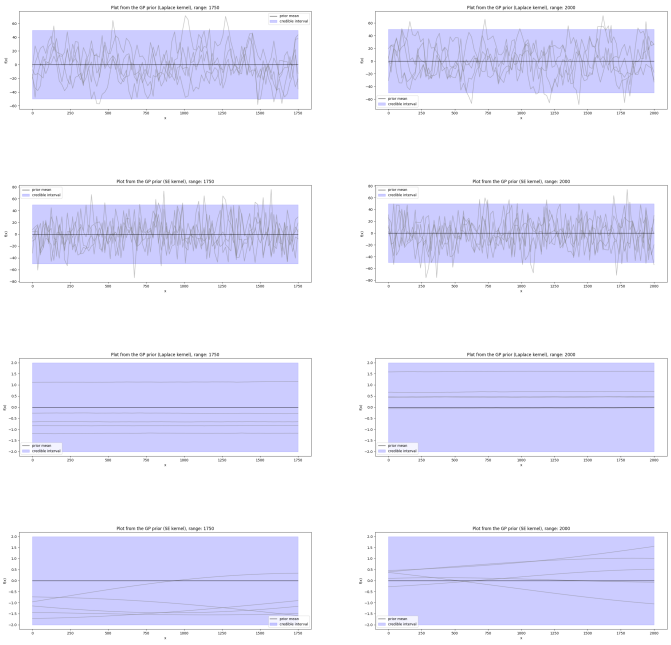
$\alpha = 100$

Figure 2: Generated samples under different values of α

(g) Applying the temperature transformation pushes probabilities away from 0.5, reducing uncertainty and producing clearer samples. However, large values of α make pixel values increasingly deterministic, driving probabilities towards 0 or 1 and reducing sample diversity. Therefore, a moderate α achieves a good balance between sharpness and diversity.

Part 2: Gaussian processes

(b) The experimental settings are presented below. The upper four images correspond to $l = 5$, whereas the lower four images correspond to $l = 2000$.



Discussion. Comparing the prior samples with $l = 5$ and $l = 2000$ shows that the length-scale dominates the behaviour of the GP prior. With $l = 5$, the sampled functions vary rapidly and exhibit rich local fluctuations, indicating high flexibility. In contrast, when $l = 2000$ (far larger than the input range), the process becomes highly correlated across the domain and the samples collapse to near-constant or very smooth trajectories, reflecting a severe loss of expressive power and potential underfitting. This highlights the need to choose l at a scale comparable to the variation in the data to balance flexibility and smoothness.

Figure 3: GP prior samples under different kernels and hyperparameter settings

(d) We present the results with different experiments settings: initial starts, training size, optimization methods, final negative likelihood and iterations.

Kernel	Init	Size	Method	Final NLL	Iter
SE	[10,25,1]	90	L-BFGS-B	236.185	20
SE	[10,25,1]	180	L-BFGS-B	419.318	18
SE	[10,25,1]	90	Nelder-Mead	235.372	130
SE	[10,25,1]	180	Nelder-Mead	426.646	111
SE	[5,40,5]	90	L-BFGS-B	251.451	16
SE	[5,40,5]	180	L-BFGS-B	431.933	17
SE	[5,40,5]	90	Nelder-Mead	239.982	156
SE	[5,40,5]	180	Nelder-Mead	451.943	132
Laplace	[10,25,1]	90	L-BFGS-B	228.626	17
Laplace	[10,25,1]	180	L-BFGS-B	439.590	15
Laplace	[10,25,1]	90	Nelder-Mead	248.980	103
Laplace	[10,25,1]	180	Nelder-Mead	438.499	83
Laplace	[5,40,5]	90	L-BFGS-B	253.814	16
Laplace	[5,40,5]	180	L-BFGS-B	435.333	20
Laplace	[5,40,5]	90	Nelder-Mead	248.105	252
Laplace	[5,40,5]	180	Nelder-Mead	432.559	107

Table 1: Negative log-likelihood results for different kernels, initialisations, optimisers, and training sizes

From Table 1, the results show that increasing the training size from 90 to 180 leads to a substantial increase in NLL, indicating that the fixed-capacity model becomes harder to fit on larger datasets. L-BFGS-B consistently converges in far fewer iterations than Nelder-Mead while achieving comparable or better NLL values, demonstrating the advantage of gradient-based optimisation. In addition, different initialisations lead to noticeable performance variations, suggesting the presence of multiple local optima. Finally, the SE kernel generally achieves slightly lower NLL than the Laplace kernel, implying that the underlying data exhibits relatively smooth structures.

(f) When sampling from the posterior GP, we plot 5 GP trajectories for each kernel by sampling 2000 points from 0 to 1750 with the same gap for each pair of points. In terms of the training process, we choose $l, \sigma, \sigma_\epsilon = [5, 25, 1]$ as our optimization start, and use L-BFGS-B as our optimizer.



(a) SE kernel

(b) Laplace kernel

Figure 4: Posterior distributions of Gaussian process models with different kernels

By choosing SE kernel, the trajectories of GP are much smoother than those use Laplace kernel, which may be due to the differentiability of SE kernel while Laplace is not differentiable at some point; The approximation by Laplace kernel has higher probability to generate extreme values, which is also indicated by the credible interval: the variance of each point estimation is generally higher than that using SE kernel; When approximating the sparse area , SE kernel is more sensible compared to the Laplace kernel, meaning that it will generate more uncertainty, but it performs really well when estimating the dense area. Laplace kernel performs worse than the SE kernel in dense area, but it seems much stable than that in sparse place.
(g) Let's discuss the MSE to end this part:

Kernel	Init θ	Optimiser	NLL	Test MSE
SE	[5,25,1]	L-BFGS-B	449.76	3.94
Laplace	[5,25,1]	L-BFGS-B	57.82	3.24

Table 2: Comparison of predictive performance between SE and Laplace kernels

Table 2 summarises the optimisation and predictive results. While the SE kernel attains a lower training NLL, the Laplace kernel achieves a lower test MSE, suggesting improved predictive accuracy despite a less favourable marginal likelihood. This indicates that the Laplace kernel provides a more suitable inductive bias for capturing the underlying structure of the time series, particularly in the presence of local irregularities and non-smooth patterns. On the other hand, the Laplace kernel demonstrates a more moderate and stable estimation across the entire test set, with reduced sensitivity to local fluctuations. This highlights a trade-off between probabilistic model fit and point prediction performance, where a model with inferior marginal likelihood can still deliver superior predictive accuracy on unseen data.

Part 3: Recovering EM from VI

We start from ELBO, that is, by fixing old parameter θ_0^{old} considering

$$\begin{aligned}
\text{ELBO}(q) &= \mathbb{E}[\log p(x,\theta,z)] - \mathbb{E}[\log q(\theta,z)] \\
&= \int q(\theta,z)\log p(x,\theta,z)d\theta dz - \int q(\theta,z)\log q(\theta,z)d\theta dz \\
&= \int q(\theta)q(z)\log p(x,\theta,z)d\theta dz - \int q(\theta)q(z)\log q(\theta,z)d\theta dz \\
&= \int q(z)\log p(x,\theta_0^{old},z)dz - \int q(z)\log q(\theta_0^{old},z)d\theta dz \\
&= \int q(z)\log p(x,\theta_0^{old},z)dz - \int q(z)\log q(z)dz + \text{constant} \\
&= -\text{KL}(q(z)||p(z|\theta_0^{old},x)) + \log p(\theta_0^{old},x) + \text{constant},
\end{aligned}$$

where all expectations are taken w.r.t the joint distribution $q(z,\theta)$. As $\text{KL} \geq 0$, then we need to set

$$q^{new}(z) = p(z|\theta_0^{old},x)$$

to maximize ELBO, which is our "E-Step".

We then update θ_0 by maximising the ELBO with respect to θ_0, while keeping $q^{new}(z)$ fixed in the joint distribution. Since the entropy term satisfies:

$$-\mathbb{E}(\log q(\theta,z)) = -\mathbb{E}_q(\log q(z)) - \mathbb{E}_\delta \log \delta(\theta-\theta_0),$$

so this term is divergent or but independent of θ_0 while $q(z)$ is fixed. Then the optimization does not depend on θ_0, which is equivalent to say

$$\theta_0^{new} = \arg\max_{\theta_0} \mathbb{E}_{q^{new}(z)} \log p(x,z,\theta_0).$$

The above procedure corresponds to the M-step of the EM algorithm.

Part 4: KL divergence as a particular case

The metric between probability measures p and q is defined as

$$D_\alpha(p\|q) = \frac{4}{1-\alpha^2}\left(1 - \int p(x)^{\frac{1-\alpha}{2}} q(x)^{\frac{1+\alpha}{2}}\, dx\right).$$

We observe that as $\alpha \to 1$ and $\alpha \to -1$, the first factor diverges to infinity, while the integral term converges to zero. Hence, the expression is of the indeterminate form $\infty \cdot 0$, and L'Hospital's rule can be applied to evaluate the limit.

Case $\alpha \to 1$. When $\alpha \to 1$, differentiating the numerator and denominator with respect to α, we obtain

$$\begin{aligned}
\lim_{\alpha\to 1} D_\alpha(p\|q) &= \lim_{\alpha\to 1} \frac{1 - \int (p(x)q(x))^{1/2} \left(\frac{q(x)}{p(x)}\right)^{\alpha/2} dx}{\frac{1-\alpha^2}{4}} \\
&= \lim_{\alpha\to 1} \frac{-\int (p(x)q(x))^{1/2} \left(\frac{q(x)}{p(x)}\right)^{\alpha/2} \log\frac{q(x)}{p(x)} \cdot \frac{1}{2}\, dx}{-\frac{\alpha}{2}} \\
&= \int q(x) \log\frac{q(x)}{p(x)}\, dx \\
&= \mathrm{KL}(q\|p).
\end{aligned}$$

Case $\alpha \to -1$. Similarly, when $\alpha \to -1$, applying L'Hospital's rule yields

$$\begin{aligned}
\lim_{\alpha\to -1} D_\alpha(p\|q) &= \lim_{\alpha\to -1} \frac{1 - \int (p(x)q(x))^{1/2} \left(\frac{q(x)}{p(x)}\right)^{\alpha/2} dx}{\frac{1-\alpha^2}{4}} \\
&= \lim_{\alpha\to -1} \frac{-\int (p(x)q(x))^{1/2} \left(\frac{q(x)}{p(x)}\right)^{\alpha/2} \log\frac{q(x)}{p(x)} \cdot \frac{1}{2}\, dx}{-\frac{\alpha}{2}} \\
&= \int p(x) \log\frac{p(x)}{q(x)}\, dx \\
&= \mathrm{KL}(p\|q).
\end{aligned}$$

Therefore, we conclude that

$$\lim_{\alpha\to 1} D_\alpha(p\|q) = \mathrm{KL}(q\|p), \qquad \lim_{\alpha\to -1} D_\alpha(p\|q) = \mathrm{KL}(p\|q),$$

which completes the proof.